

DayPilot engineering audit

DayPilot engineering audit

7 September 2026

Repository files only. `.env` was not read. Production data was not queried.

Live stack: Next.js 16, NestJS + Prisma, Supabase Auth, Flutter testers, SwiftUI rewrite.

Not the prompt stack: Vite, React 18, Expo.

Confidence: high on topology and commands; medium on live Railway/TLS timing.

DayPilot repository audit

Date: 7 September 2026

Branch: audit/engineering-baseline

Confidence: high on topology and commands; medium on live Railway/TLS timing.

Sources: repository files only. `.env` was not read.

This audit describes what is in the tree today. It does not follow the prompt's Vite / React 18 / Expo assumptions.

Architecture map

```

Next.js web (apps/web)
├─ Supabase Auth / RLS / Storage / Edge Function pilot-brief
├─ Nest JWT via POST /auth/supabase-exchange
│   └─ events, billing, AI suggestions
│       └─ calendar-connections (Google, Outlook, EventKit ingest)

Flutter iOS (daypilot_flutter)    testers, TestFlight
SwiftUI iOS (apps/ios)            rewrite target
├─ Supabase Auth
├─ EventKit on device
└─ Nest (exchange + events + EventKit sync)

Prisma Postgres                  events + CalendarConnection + Nest users
Supabase Postgres                profiles, tasks, booking, friends, pilot_briefs

```

Shared TypeScript types live in `packages/lib/src/calendar/types.ts`. `@daypilot/ui` is still a stub.

Surfaces

Surface	Path	Role
Web	apps/web	Marketing + authenticated app. Next.js 16, React 19, Tailwind 4
API	apps/api	NestJS 11. Source of calendar OAuth and synced events
Flutter	daypilot_flutter/	Tester daily driver. Maintenance only (ADR-004)
SwiftUI	apps/ios/	Product iOS target. Repositories only for Graph/EventKit/Supabase
Legacy	archive/legacy-v1	Old Vite + ASP.NET + Edge Functions. Reference only

There is **no** React Native / Expo app.

Major dependencies

Area	Packages
Web	next@16.1.6, react@19.2.3, @supabase/ssr, @tanstack/react-query, zustand, axios, socket.io-client
API	@nestjs/*@11, prisma@5.22, googleapis, @microsoft/microsoft-graph-client, jose, openai, @anthropic-ai/sdk, stripe
Root	pnpm@9.14.2, turbo@2.3, Node >=20 (CI uses 20)

Technical debt

1. Dual production databases (Prisma events vs Supabase product tables).
2. Calendar OAuth tokens stored as plaintext strings on CalendarConnection (`prisma/schema.prisma`).

3. Nest JWT access/refresh kept in web localStorage (apps/web/src/lib/supabase/auth.ts).
 4. Empty @daypilot/ui; @daypilot/lib lint script is a no-op.
 5. Web has **no** package.json test script. Flutter already has ~15 Dart tests (daypilot_flutter/test/, including calendar_connection_ui_test.dart). Main CI still does not run API Jest or Flutter tests.
 6. Two iOS codebases until SwiftUI reaches parity.
 7. README and docs/architecture/REPOSITORY_AUDIT.md (21 July 2026) still describe a Vite/Expo target that ADR-003/004 rejected.
 8. iCloud CalDAV (POST /calendar-connections/apple/connect) is dormant; product path is EventKit.
 9. No live ICS subscription provider (legacy archive only).
 10. Local main was one commit ahead of origin/main at audit start (checkpoint before checking out main).
-

Security risks

Severity	Finding
HIGH	Prisma accessToken / refreshToken plaintext
HIGH	Web Nest JWTs in localStorage (XSS)
HIGH	Nest user is email-only; Apple relay email can split one person into two Nest users
MEDIUM	Flutter Nest JWTs in SharedPreferences (not Keychain)
MEDIUM	Dual auth identity (Supabase UUID vs Nest cuid) linked only by email
MEDIUM	Pilot Brief Edge Function CORS is * (supabase/functions/pilot-brief/index.ts)
LOW	.env exists locally and is gitignored — do not commit

No client-side service-role key was found in application source. .env.example documents anon + Nest secrets only.

Performance concerns

- Auth enrich is backgrounded (good). Google SSO auto-connect can hard-redirect to OAuth (maybeAutoConnectCalendars).
 - Calendar views are custom day/week/month components — no virtualization library found.
 - Flutter and SwiftUI both present; shipping both increases QA cost.
 - EventKit uploads were previously 413-limited; chunking exists (fix/eventkit-413-and-session).
-

Testing gaps

Area	What exists
API	Jest specs for Graph client, OAuth callback, EventKit reconcile/dedupe, CalDAV password shape, health controller
Web	New <code>calendar-connection-ui.spec.ts</code> on this branch; no <code>package.json</code> test script
Flutter	~15 Dart tests including <code>calendar_connection_ui_test.dart</code> ; Flutter CI runs them. Main <code>ci.yml</code> does not
SwiftUI	<code>apps/ios/Tests/DayPilotCoreTests</code>
E2E	No Playwright/Cypress in the live tree
RLS	No automated policy tests

Validation commands that exist

Step	Command	Notes
Format	<code>pnpm --filter @daypilot/api run format</code>	API only (Prettier)
Lint	<code>pnpm lint (turbo)</code>	<code>@daypilot/lib</code> prints a stub
Typecheck	<code>none named</code>	<code>next build / nest build</code> typecheck as they compile
Unit tests	<code>pnpm --filter @daypilot/api test</code>	API only
Web build	<code>pnpm --filter @daypilot/web run build</code>	
Mobile	<code>flutter analyze / flutter test</code>	Flutter CI; Android build exists but Android product work is frozen

Top 10 engineering priorities

1. Keep Identity Authentication separate from Calendar Authorization in every UI (Apple SSO $\neq$ EventKit).
2. Encrypt or vault calendar tokens at rest.
3. Move web Nest tokens off `localStorage`.
4. Honest EventKit status when iOS permission is denied or stale.
5. One event read model for web/iOS so Prisma/Supabase split is explicit in the UI.
6. Web unit tests for auth mapping, connection UI, and timezone helpers.
7. Add API tests to CI (currently build+lint only).
8. Finish `api.daypilot.co` TLS cutover or stop advertising it as live.
9. SwiftUI parity for calendar + sync + auth so Flutter can be retired.
10. Stop writing new features on Flutter (ADR-004).

Git state at audit

- Default remote branch: origin/main
- Working branch for this write-up: audit/engineering-baseline
- Existing fix branches already landed conceptually: EventKit 413, null availability, Graph /v1.0/v1.0, connection status UI # DayPilot architecture (actual)

Date: 7 September 2026

Confidence: high

```
Web App (Next.js)
  ↓
Shared services (thin): @daypilot/lib types, per-app API clients
  ↓
Auth / Calendar / Scheduling / AI
  ↓
Supabase (Auth, RLS product tables, Storage, Edge Function pilot-brief)
  +
NestJS (JWT, events, OAuth, Graph, EventKit ingest, billing, AI suggest)
  ↓
Prisma Postgres                Supabase Postgres

Mobile testers (Flutter)
  ↓
Supabase Auth + EventKit + Nest adapters

Mobile target (SwiftUI)
  ↓
Repositories only → Nest + EventKit + Supabase
```

This is **not** Vite → shared RN services → Supabase-only.

Frontend architecture

apps/web uses the App Router:

- Public: /, login, signup, privacy, auth/callback
- Authenticated (app): home, calendar, sync, integrations, settings, billing, tasks, notes, focus, meetings, friends, booking-links, pilot-brief
- Providers: AuthProvider, ThemeProvider, React Query
- Calendar UI: WeekCalendarView, DayTimeline, EventModals, HomeDashboard
- Auth unlocks from Supabase session; Nest exchange is enrichment (AuthProvider.tsx)

packages/lib exports CalendarProviderAdapter and normalized event types. Nest and Flutter do **not** yet implement that interface as concrete classes. Google/Outlook/Apple logic lives in Nest services.

Mobile architecture

Client	Bundle (docs)	Allowed to call
Flutter	com.dekuworks.daypilot	Supabase, Nest, EventKit (maintenance)
SwiftUI	com.dekuworks.daypilot.swift	Repositories wrapping Nest + EventKit + session store

SwiftUI: DayPilotCore (NestAPIClient, EventKitPayloadMapper), app features under apps/ios/App/DayPilot/Features/.

ADR-004: no new Flutter features; do not delete Flutter until SwiftUI covers daily calendar + sync + auth.

Auth flow

```
User → Supabase Auth (email, Google, Apple, Microsoft)
      → web/iOS session
      → POST /auth/supabase-exchange (Supabase access token)
      → Nest JWT (localStorage on web; in-memory/session store on iOS)
      → JwtAuthGuard on /events and /calendar-connections
```

Identity providers do **not** grant calendar scopes by themselves.

Google/Microsoft SSO may *trigger* a second calendar OAuth via maybeAutoConnectCalendars. Apple SSO never does.

Calendar connection flow

```
Connect Provider
  ↓
Authorization
  Google/Outlook: Nest OAuth (secrets on server)
  Apple: EventKit permission on device (CalDAV ASP dormant)
  ↓
Token / Native Permission
  ↓
Calendar Discovery
  ↓
Event Fetch (Google Calendar API, Graph calendarView, EventKit payload)
  ↓
Normalization (source + externalId on Prisma Event)
  ↓
DayPilot DB (Prisma)
  ↓
Unified calendar via GET /events
```

Apple web visibility depends on iOS posting POST /calendar-connections/apple/eventkit/sync.

Event synchronization flow

1. OAuth providers: `syncConnection` in `calendar-connections.service.ts`
 2. EventKit: `eventkit-sync.service.ts` (chunked, deduped against Google/Microsoft device calendars)
 3. Clients poll or use the Socket.IO events gateway
 4. Duplicates: `packages/lib` helpers (`detectDuplicateCalendar`, `generateEventFingerprint`)
-

AI scheduling flow

1. **Suggest schedule** — Nest `apps/api/src/ai/` (OpenAI / Anthropic / compatible). Web: `SuggestScheduleCard`.
2. **Pilot Brief** — Edge Function `supabase/functions/pilot-brief` reads Nest events + Supabase tasks, optional model, always has a fallback. Web page can regenerate via `functions/v1/pilot-brief`.

Keys stay server-side (Nest env or Edge secrets).

Supabase data model (product)

Migrations under `supabase/migrations/`: profiles, calendars, events (Supabase copy), tasks, categories, attendees, share/booking links, preferences, reminders, orgs/members, contacts, friends, notifications, avatars bucket, `pilot_briefs` / `messages`.

RLS is the access control for those tables. This store is **not** the Nest event table.

Prisma data model (calendar API)

User, RefreshToken, CalendarConnection (tokens), ExternalCalendar, Event (source: `native|google|outlook|apple|apple_eventkit|booking`), Task stub, Organization, Team, Subscription, AuditLog, sync logs.

Edge Function responsibilities

Function	Role
<code>supabase/functions/pilot-brief</code>	Daily brief + chat; calls Nest for events

Archived Edge Functions (Google OAuth, Stripe, email) live under `archive/legacy-v1` and are not the live path.

Provider-specific adapters

Provider	Implementation	Interface?
Google	googleapis in Nest calendar-connections	No class implementing CalendarProviderAdapter
Microsoft	graph-client.ts	Same
Apple EventKit	iOS native + Nest ingest	APPLE_EVENTKIT_CAPABILITIES in lib
Apple CalDAV	icloud-caldav.ts	Dormant product path
ICS	Legacy booking download only	Not a live provider

Nest maps outlook ↔ microsoft at the API boundary (packages/lib comment). # Authentication audit

Date: 7 September 2026

Confidence: high

Identity Authentication vs Calendar Data Authorization

Identity Authentication

Supabase Auth: email/password, magic link, Google, Apple, Microsoft

Proves who the DayPilot user is.

Tokens: Supabase session (+ Nest JWT after exchange).

Calendar Data Authorization

Separate grant to read/write a provider calendar.

Google: Nest OAuth (`calendar.events`, `calendar.readonly`) – not granted by Google SSO

Outlook: Nest OAuth, or Microsoft SSO `provider\_token` when login requested

`Calendars.ReadWrite`

Apple: EventKit permission on the device (not Sign in with Apple)

Sign in with Apple is not EventKit and is not iCloud CalDAV.

The web handoff page already states this (apps/web/src/app/app/integrations/apple-calendar/page.tsx).

mapAppleEventKitUi comment: “Profile/Sync Apple row = EventKit, never Sign in with Apple.”

If a tester signs in with Apple and sees an empty calendar, that is expected until the iOS app grants EventKit and syncs.

Web

Topic	Implementation
Session SOT	Supabase (AuthProvider)
Nest bridge	exchangeNestSession → localStorage keys accessToken / refreshToken / user
Unlock	UI unlocks when Supabase session exists; Nest/profile enrich is background
Deadlock guard	No await inside onAuthStateChange
SSO	loginWithGoogle, loginWithMicrosoft, loginWithApple({ next })
Protected routes	App shell + auth callback
Auto calendar	maybeAutoConnectCalendars — Google/Microsoft identities only

Known issue: Apple/iCloud sign-in without calendar events

Reproduced in product terms (not a crashed API):

1. User completes Sign in with Apple (identity succeeds).
2. No EventKit permission is requested on web.
3. maybeAutoConnectCalendars skips Apple identities.
4. GET /events has no apple_eventkit rows until iOS sync.

This is a **category error** if the UI implies Apple SSO “connected the calendar”. Sync/Integrations copy must keep saying “open the iPhone app”.

Mobile

Client	Auth	Persistence
Flutter	Supabase + POST /auth/supabase-exchange (NestApiSession)	Nest JWTs in SharedPreferences , not secure storage
SwiftUI	Same exchange via NestAPIClient	InMemorySessionStore; no Nest refresh-token rotation

Account linking: Nest resolves the user **by email** at exchangeFromSupabaseAccessToken. There is no Supabase user.id on Prisma User. Apple Hide My Email (@privaterelay.appleid.com) vs Gmail can create **two Nest users**. No link-accounts UI.

Token refresh and logout

- Supabase refreshes its own session.
- Nest refresh tokens are hashed in Prisma (RefreshToken).
- Logout should clear both stores (clearNestSession + Supabase signOut). If only one is cleared, the other client may still look signed in.

Recommendations

- 1. After Apple SSO, land on /sync with an EventKit-only banner (do not start CalDAV).
- 2. Move Nest JWTs to httpOnly cookies on web; use secure storage on Flutter (not SharedPreferences).
- 3. Persist SwiftUI sessions in the Keychain, not memory only. Link Nest users by Supabase sub, not email alone.
- 4. Never reuse Sign in with Apple as a calendar health signal. # Calendar sync audit

Date: 7 September 2026

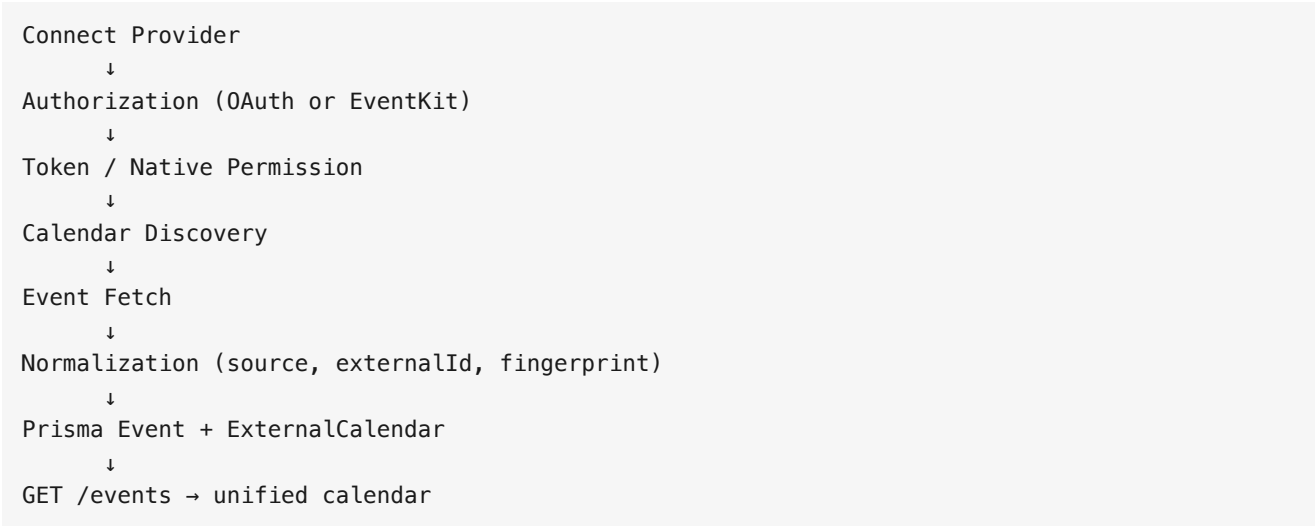
Confidence: high on code paths; medium on production rate limits (not exercised here).

Provider matrix

Provider	Live?	Connect	Fetch	Notes
Google Calendar	Yes	Nest OAuth	Google Calendar API	Auto-connect after Google SSO
Outlook / Microsoft Graph	Yes	Nest OAuth or from-token	calendarView	Must use Graph host + defaultVersion: v1.0 only once
Apple EventKit	Yes	iOS permission + POST sync	Device → Nest	Web is a cloud copy, read-oriented
Apple CalDAV / iCloud ASP	Dormant	POST /apple/connect	CalDAV	Not the product path
ICS	No	—	—	Legacy booking .ics download only
Local device	iOS only	EventKit	Local UI	Also uploaded as apple_eventkit

There is **no** ICS subscription adapter in the live Nest module.

Flow



Issues already fixed on historical branches

- Graph /v1.0/v1.0 (fix/shared-avatar-and-graph-v1)
 - EventKit 413 large uploads (fix/eventkit-413-and-session)
 - Null EventKit availability crash (fix/apple-calendar-null-availability)
 - Honest Google expired vs Apple EventKit status (fix/calendar-connections-status-ui)
-

Remaining defects

HIGH — Apple SSO $\neq$ calendar access

Identity success does not populate events. Web/iOS must keep the two states separate. `mapAppleEventKitUi` still treats any EventKit **connection row** as healthy, even if `calendarStatus` is `denied` / `permission_required` / `restricted`.

HIGH — Tokens at rest

`CalendarConnection.accessToken` and `refreshToken` are plaintext.

MEDIUM — Dual event stores

Supabase events (RLS) vs Prisma Event. Clients that read the wrong store look “out of sync”.

MEDIUM — Recurrence / timezone / all-day on Google and Outlook

Prisma has `allDay`, `timezone`, `recurrenceRule`. EventKit ingest writes them. **Google and Outlook sync do not** — Google uses `singleEvents: true` (expanded instances, no RRULE stored) and import omits `timezone/all-day`. Unified UI can lie about all-day Outlook/Google rows.

Web `apps/web/src/lib/events.ts` prefers Nest when a Nest JWT exists, else falls back to Supabase events. Those two tables can disagree (KI-001).

MEDIUM — Token expiry UX

OAuth rows can show `expired` / `needs_reconnect`. EventKit rows do not use that vocabulary.

LOW — Rate limits

No explicit Google/Graph backoff module found beyond provider SDK defaults.

LOW — Stale EventKit rows

`shouldDeleteStaleEventKitRow` is unit-tested; still depends on iOS sending a full snapshot.

Adapter interface

CalendarProviderAdapter in packages/lib is the target shape. Nest still uses a single CalendarConnectionsService with provider if branches. A class-per-provider refactor would touch every OAuth path. **Do not do that in this pass** — duplication is real, but destabilising working Google/Outlook is a worse trade.

Phase 7 fix (this branch)

mapAppleEventKitUi now treats EventKit calendarStatus of denied, restricted, permission_required, unavailable, or error (and syncStatus === error) as needsAttention, with copy that asks the user to allow Calendar access on iPhone — not to sign in with Apple again.

Regression: apps/web/src/lib/calendar-connection-ui.spec.ts. # Supabase / database audit

Date: 7 September 2026
Confidence: high on migration list and Prisma overlap; RLS details summarised from filenames and existing docs. Production was not queried.

Do not run destructive migrations.

Two Postgres databases

Store	Used for	Migrations
Supabase	Auth users, profiles, tasks, booking, orgs, friends, notifications, Pilot Brief, avatars	supabase/migrations/
Prisma / Nest	Nest users, events, calendar connections, subscriptions, audit	prisma/

ADR-001 said “Supabase primary”. Live 15 August 2026 audit reversed that for calendar: Graph/EventKit/JWT stay on Nest.

Supabase migrations (filenames)

- 20260602190001_initial_schema.sql
- 20260602190002_core_calendar_features.sql
- 20260602190003_organizations.sql
- 20260602190004_booking_links.sql
- 20260602190005_mvp_persistence.sql
- 20260602190006_realtime_and_rsvp_policies.sql
- 20260721180000_rebrand_schema_gaps.sql
- 20260721200000_profile_name_username.sql
- 20260721210000_contacts.sql
- 20260721220000_notifications_realtime.sql
- 20260721230000_fix_org_members_rls_recursion.sql

- 20260721240000_tasks_urgent_priority.sql
- 20260726030000_friends_social.sql
- 20260815150000_avatars_storage.sql
- 20260816010000_workspace_unique_color.sql
- 20260816020000_pilot_brief_messages.sql

20260721230000_fix_org_members_rls_recursion.sql shows RLS has already bitten once (recursive org-member policies).

Ownership

- Product tables: auth.uid() via RLS (user-owned profiles, tasks, booking links).
- Nest events: Event.userId → Prisma User created/linked at supabase-exchange.
- Shared calendars / booking: Supabase share and booking-link tables; not the same as Prisma source: booking until those paths meet.

Edge Functions

Path	Notes
supabase/functions/pilot-brief/index.ts	User JWT for the caller; SUPABASE_SERVICE_ROLE_KEY server-side to read/write briefs. Also merges Nest /events. CORS *.

No service-role usage in apps/web, apps/api, or Flutter clients. Do not add it to a browser bundle. integration_connections.encrypted_credentials_reference exists in Supabase but is **not** wired to Nest token storage.

Risks

Severity	Finding
HIGH	Event truth split across two DBs
HIGH	Nest calendar tokens plaintext
MEDIUM	Org RLS recursion history — re-test any new org policy
MEDIUM	Supabase events table may confuse new agents into writing the wrong store
LOW	packages do not generate Supabase types in CI

Indexes, FKs, and N+1 were not measured against production. Prisma CalendarConnection has userId indexes; Event uniqueness should stay on (userId, source, externalId) — verify before adding writes. # Validation baseline

Date: 7 September 2026

Commands discovered from package.json and CI. Results recorded below after a local run. Failures are not suppressed.

Desired flow vs reality

Format	API Prettier only
↓	
Lint	turbo pnpm lint (web eslint, api eslint, lib stub)
↓	
Typecheck	no script – included in builds
↓	
Tests	API Jest only
↓	
Web Build	next build
↓	
Mobile	flutter analyze / flutter test (not run in this pass unless noted)

Commands

```
pnpm --filter @daypilot/api run format          # writes
pnpm --filter @daypilot/api exec prettier --check "src/**/*.ts"
pnpm lint
pnpm --filter @daypilot/api test
pnpm --filter @daypilot/api run build
pnpm --filter @daypilot/web run lint
pnpm --filter @daypilot/web run build
```

CI (.github/workflows/ci.yml): pnpm install --frozen-lockfile → pnpm db:generate → pnpm run build → pnpm run lint. **Tests are not in CI.**

Run log

Command	Result
pnpm --filter @daypilot/api test	Pass — 7 suites, 43 tests
pnpm --filter @daypilot/api run lint	Fail — 2 errors, 91 warnings (existing). Script uses --fix; that rewrite was reverted and is not part of this work
Root pnpm lint / web build	Not treated as green until recorded separately
Web unit tests	No script
Flutter	flutter test exists (~15 files). Flutter CI runs it. Main ci.yml does not. Not re-run in this pass

Do not hide the API lint failures. They predate this audit. # Performance audit

Date: 7 September 2026

Confidence: medium (static review, no profiler run).

Web

- Auth: session unlock is fast-path; Nest exchange timed at 2.5s (`NEST_EXCHANGE_TIMEOUT_MS`). Good.
- `maybeAutoConnectCalendars` can redirect the whole window to Google OAuth after first SSO — feels like a hang if the user expected the home calendar.
- Calendar widgets (`WeekCalendarView`, `DayTimeline`) render event lists in-component. No react-window / virtualization dependency. Large synced days will get expensive.
- Duplicate fetches: Auth enrich + calendar pages each call connections/events. React Query is present — use it for connections/events consistently.
- Bundle: Next 16 + Sentry + stripe + socket.io-client. Acceptable; do not add another calendar kit without measuring.

API / sync

- EventKit ingest is chunked after the 413 incident.
- Graph `calendarView` is range-bounded — keep windows tight.
- No explicit sync batching/backoff module.
- `GET /metrics` is in-memory request counts only (`docs/OBSERVABILITY.md`).

Mobile

- Two runtimes (Flutter + SwiftUI) on a device increase install/QA cost, not runtime of a single app.
- Flutter startup/offline: existing caching mentioned in older audits; not re-profiled.
- SwiftUI `InMemorySessionStore` forces re-login (product issue more than FPS).

Supabase

- Pilot Brief function merges Nest events and tasks in Deno. Keep payloads to “today” only (already implied by brief date).
- Avatars bucket is the right place for photos — do not embed large SSO images in every event query.

Recommendations (not implemented here)

1. Virtualise day/week event lists above a few hundred events.
2. Cache `listConnections` + `getEventKitStatus` in React Query with a shared key.
3. Do not auto-redirect Google OAuth on every new browser profile without a Sync CTA. # AI feature audit

Date: 7 September 2026

Confidence: high on locations; medium on production model/cost (env not read).

Surfaces

Feature	Where	Model access
Schedule suggestions	Nest apps/api/src/ai/ + web SuggestScheduleCard	OPENAI_* / ANTHROPIC_* on the API
Pilot Brief	supabase/functions/pilot-brief + web /pilot-brief	Optional; fallback always works
Flutter / SwiftUI Pilot	Read the same pilot_briefs row	Do not rebuild the router (15 Aug audit)

Prompt construction

Pilot Brief builds a structured BriefContent (summary, counts, suggestions, conflicts, focus windows, follow-ups) and a source: "ai" | "fallback". Events are reduced to { title, start, end } before merge (eventKey). That is the right data-minimisation direction — do not send descriptions, attendees, or conferencing notes unless a later spec requires it.

Nest AI suggest is a separate path (suggest-schedule.dto.ts). Treat it as server-only.

Risks

Topic	Status
Hallucination	Fallback path exists; UI should show source
Permissions	Function expects a user JWT; still uses CORS *
API failure	Fallback brief must remain
Cost	No quota/entitlement wrapper seen in the function header
Validation	Response shaped in-function; clients should not render raw model text unchecked
Secrets	Keys in Edge/Nest env — never NEXT_PUBLIC_

Do not

- Log event titles to third-party analytics without a privacy review.
- Send the full Prisma event row (tokens, emails of attendees) to the model.
- Rebuild Pilot Brief as a SwiftUI-only model call. # Mobile audit

Date: 7 September 2026

Confidence: high on split of responsibilities; medium on Flutter internals not fully re-walked this session.

Two clients

	Flutter <code>daypilot_flutter/</code>	SwiftUI <code>apps/ios/</code>
Role	Tester daily driver (TestFlight)	Product target (ADR-004)
Auth	Supabase + Nest exchange	Same, via <code>NestAPIClient</code>
Calendar	EventKit + Nest events	EventKit mapper + Nest sync DTO
New features	Frozen except crash/sync/auth/TestFlight	Allowed, through repositories

There is **no** Expo / React Native app. Do not scaffold `apps/ios` as RN.

EventKit vs Sign in with Apple

- EventKit: OS calendar permission. Required to see iCloud/Google-on-device/subscribed calendars locally and to POST the cloud copy.
- Sign in with Apple: Supabase identity only.
- Deep link for setup: `com.daypilot.daypilot://integrations/apple-calendar` (web handoff page).

If Sign in with Apple succeeds and the calendar is empty, check EventKit permission — not the Apple SSO token.

Other mobile topics

Topic	Notes
Auth persistence	SwiftUI <code>InMemorySessionStore</code> is a gap vs Flutter
Widgets	Flutter shipped home/lock-screen widgets (commit 74d012d). ADR-004 says do not start new WidgetKit work on SwiftUI in the 90-day slice
Notifications	Flutter wiring exists historically; confirm permission copy distinguishes calendar vs alerts
Background sync	EventKit ingest is app-driven POST, not a documented server pull of iCloud
Offline	Flutter caching mentioned in older docs; SwiftUI not there yet
iPhone / iPad	Flutter is the QA target; Android is frozen
Secrets	<code>dart-define.json</code> gitignored; no Google/Microsoft client secrets in the iOS bundle (ADR-004)

Recommendations

1. Keychain session store for SwiftUI.
2. Surface EventKit denied on web (`mapAppleEventKitUi`).
3. Keep Flutter TestFlight alive until SwiftUI parity.
4. Do not add Expo.